

Advanced Machine Learning

DATA 642

Lecture 10 — Clustering and *K*-means

*Figures and notes are from the book Mathematics for Machine Learning by A. Aldo Faisal, Cheng Soon Ong, and Marc Peter Deisenroth, Machine Learning A Bayesian and Optimization Perspective, Sergios Theodoridis, Elsevier, and Hands-On Machine Learning with Scikit-Learn Keras and TensorFlow, Aurelien Geron, O'Reilly

Central idea

Clustering refers to the process of grouping similar instances together in a dataset based on certain criteria. It is a fundamental technique in data analysis and machine learning with various applications:

- **Customer Segmentation:** Clustering helps in dividing customers into distinct groups based on their behavior, purchases, or interaction patterns. This segmentation aids in tailoring products and marketing strategies to better meet the needs of each customer segment.
- **Data Analysis:** Before delving into detailed analysis, identifying clusters within a dataset helps in understanding the underlying patterns and structures. It simplifies the analysis by allowing examination of clusters separately.
- **Dimensionality Reduction:** Clustering can be used as a technique to reduce the dimensionality of a dataset by identifying and representing clusters instead of individual data points, especially useful for visualization and analysis of high-dimensional data.

- **Anomaly Detection:** Unusual instances that do not fit well into any cluster can be identified as anomalies or outliers. This is particularly useful in detecting defects in manufacturing processes, fraud detection, or identifying unusual behavior in various applications.
- **Semi-Supervised Learning:** In scenarios where labeled data is scarce, clustering can be used to propagate labels from a few labeled instances to the entire cluster, thereby augmenting the available labeled data and improving the performance of subsequent supervised learning algorithms.
- **Image Segmentation:** Clustering pixels based on color similarity enables the segmentation of images. By replacing each pixel's color with the mean color of its cluster, the number of distinct colors in the image can be reduced, aiding in object detection and tracking tasks.

K -Means

The K -Means algorithm is a simple algorithm capable of clustering a dataset very quickly and efficiently, often in just a few iterations. It was proposed by Stuart Lloyd at the Bell Labs in 1957 as a technique for pulse-code modulation, but it was only published outside of the company in 1982, in a paper titled "Least square quantization in PCM".

- The K -means procedure results from a simple and intuitive mathematical problem. Let $C_1, C_2, \dots, C_K$ denote sets containing the indices of the observations in each cluster. These sets satisfy
 1. $C_1 \cup C_2 \cup \dots \cup C_k = \{1, \dots, N\}$, each observation belongs to at least one of the K clusters.
 2. $C_i \cap C_j \neq \emptyset$ for all $i \neq j$, non-overlapping

- In K -means clustering a "good" clustering is one for which the within cluster variation is as small as possible. The within cluster C_k is a measure $W(C_k)$ of the amount by which the observations within a cluster differ from each other. Hence, we want to solve

$$\min_{C_1, C_2, \dots, C_K} = \left\{ \sum_{k=1}^K W(C_k) \right\} \quad (1)$$

We want to partition the observations into K clusters such that the total within in-cluster variation, summed over all K clusters is as small as possible.

A typical choice for $W(C_k)$ is the squared Euclidean distance. This is we define

$$W(C_k) = \frac{1}{|C_k|} \sum_{i, i' \in C_k} \sum_{j=1}^P (x_{ij} - x_{i'j})^2, \quad (2)$$

where $|C_k|$ is the number of observations within a cluster. Sum of all of the pairwise squared Euclidean distances between the observations in the k th cluster, divided by the total number of observation in the k th cluster.

Algorithm

The iterative process of the K-Means algorithm involves updating cluster centroids and reassigning data points to clusters until convergence. Here's a breakdown of the iterative steps:

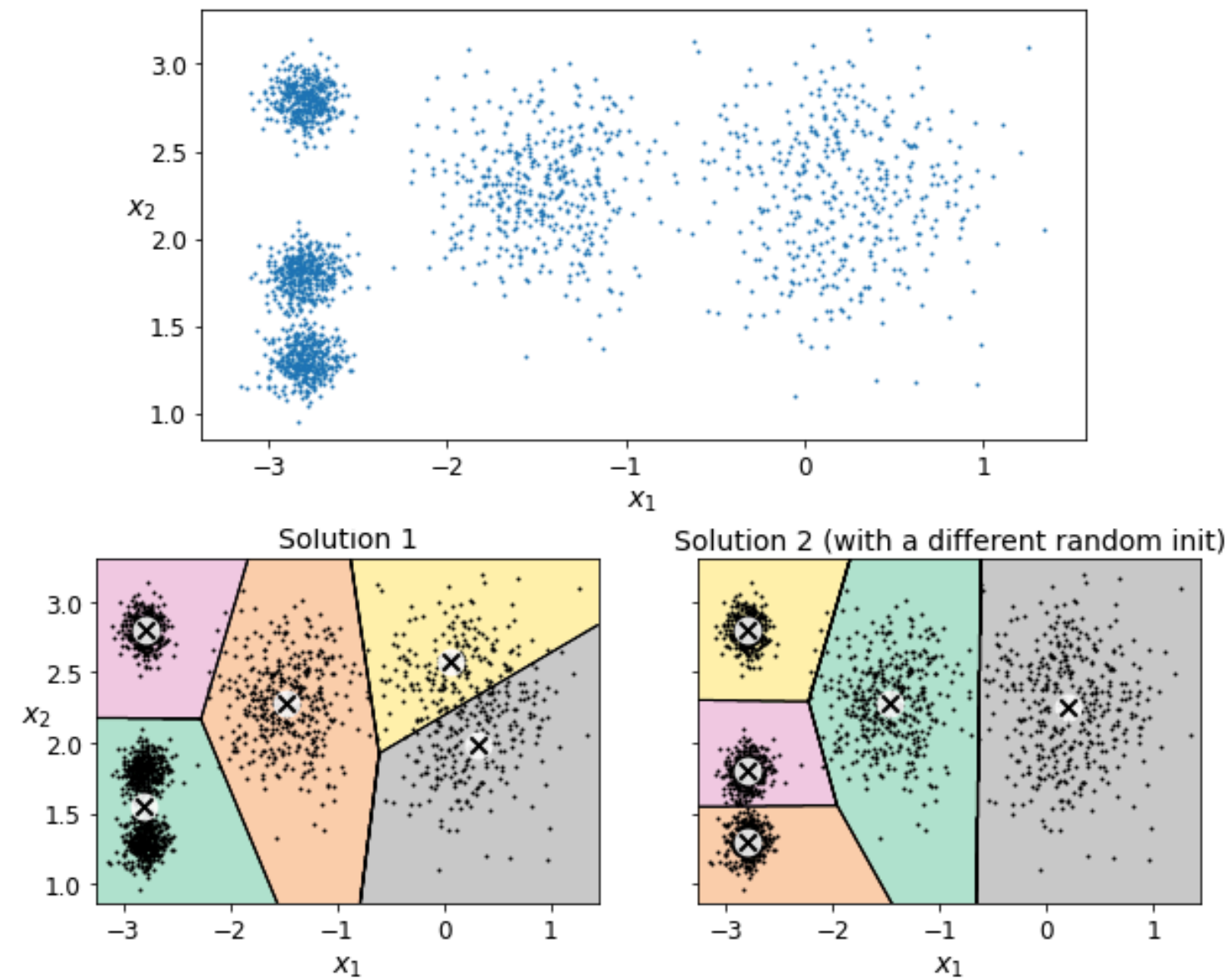
1. **Initialization:** The algorithm starts by initializing K cluster centroids. Common initialization methods include randomly selecting data points as centroids or using more sophisticated techniques like k-means++.
2. **Assignment Step:** In this step, each data point is assigned to the nearest cluster centroid based on its distance. Typically, the Euclidean distance metric is used, but other distance metrics can also be employed. This assignment step partitions the data into clusters.
3. **Update Step:** After assigning data points to clusters, the algorithm recalculates the centroids of each cluster. The new centroid of each cluster is computed as the mean of all data points assigned to that cluster. This step aims to find the center of each cluster.

4. **Convergence Check:** The algorithm checks for convergence by evaluating whether the cluster centroids have changed significantly compared to the previous iteration or if the assignments of data points to clusters have stabilized. If convergence criteria are met, the algorithm terminates; otherwise, it proceeds to the next iteration.
5. **Iterative Optimization:** Steps 2 to 4 are repeated iteratively until convergence is achieved. At each iteration, the algorithm aims to minimize the objective function, which represents the total within-cluster variation, by updating cluster centroids and reassigning data points to clusters.

Important Notes

The result of running the K-Means algorithm may indeed vary each time it's executed due to its sensitivity to initial conditions. Here are a few reasons why:

1. **Initialization:** K-Means begins by randomly selecting initial cluster centroids. Since this step is random, different initializations can lead to different cluster assignments and centroid positions.



2. **Local Optima:** K-Means aims to minimize the within-cluster variation, but it's possible to converge to a local optimum rather than the global optimum. Different initializations may result in convergence to different local optima, leading to varying final cluster configurations.
3. **Convergence Criteria:** The algorithm terminates when convergence is reached, typically when there's no significant change in cluster assignments or centroids. However, convergence may occur at different stages during each run, resulting in slightly different outcomes.

Centroid initialization methods

Instead of initializing the centroids entirely randomly, it is preferable to initialize them using the following algorithm, proposed in a [2006 paper](#) by David Arthur and Sergei Vassilvitskii:

- Take one centroid c_1 , chosen uniformly at random from the dataset.
- Take a new center c_i , choosing an instance $\mathbf{x}_i$ with probability: $D(\mathbf{x}_i)^2 / \sum_{j=1}^m D(\mathbf{x}_j)^2$ where $D(\mathbf{x}_i)$ is the distance between the instance $\mathbf{x}_i$ and the closest centroid that was already chosen. This probability distribution ensures that instances that are further away from already chosen centroids are much more likely be selected as centroids.
- Repeat the previous step until all k centroids have been chosen.

The rest of the K-Means++ algorithm is just regular K-Means. With this initialization, the K-Means algorithm is much less likely to converge to a suboptimal solution, so it is possible to reduce `n_init` considerably. Most of the time, this largely compensates for the additional complexity of the initialization process.

To set the initialization to K-Means++, simply set `init="k-means++"` (this is actually the default):

Accelerated K-Means

The K-Means algorithm can be significantly accelerated by avoiding many unnecessary distance calculations: this is achieved by exploiting the triangle inequality (given three points A, B and C, the distance AC is always such that $AC \leq AB + BC$) and by keeping track of lower and upper bounds for distances between instances and centroids (see this [2003 paper](#) by Charles Elkan for more details).

To use Elkan's variant of K-Means, just set `algorithm="elkan"`. Note that it does not support sparse data, so by default, Scikit-Learn uses `"elkan"` for dense data, and `"full"` (the regular K-Means algorithm) for sparse data.

Evaluation of K -means

1. **Inertia:**

- Measures the compactness of clusters by calculating the within-cluster sum of squares (WCSS).
- Lower inertia indicates tighter clusters.

2. **Silhouette Score:**

- Quantifies the separation between clusters and cohesion within clusters.
- Ranges from -1 to 1, where higher values indicate better clustering.

3. **Elbow Method:**

- Helps select the optimal number of clusters (K).
- Involves plotting inertia against the number of clusters and identifying the "elbow point" where the rate of decrease slows down.

4. Adjusted Rand Index (ARI):

- Measures similarity between two clustering solutions.
- Ranges from -1 to 1, with higher values indicating greater similarity.

5. Visual Inspection:

- Qualitative assessment through scatter plots or other visualizations.
- Provides insights into the structure and coherence of clusters.

6. Cross-Validation:

- Evaluates generalization performance on unseen data.
- Techniques include holdout validation and k-fold cross-validation.

Limits of K-Means

Moreover, K -Means does not behave very well when the clusters have varying sizes, different densities, or non-spherical shapes.

